

Ollama Docker Setup Documentation

Install Ollama in Docker, download lightweight models, run API tests, and make the service start automatically after reboot.

Prepared for a local Ubuntu/Linux robot voice-command workflow using Ollama models such as qwen2.5:0.5b, qwen2.5:1.5b, and llama3.2:1b.

Recommended default for low-latency command classification: qwen2.5:0.5b.

Quick Start: Minimal Working Setup

Use this if Docker and GPU support already work on your machine. This removes any broken existing Ollama container but keeps the named model volume unless you delete the volume separately.

```
docker rm -f ollama

docker run -d \
  --name ollama \
  --restart unless-stopped \
  --gpus all \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
  ollama/ollama

docker exec -it ollama ollama pull qwen2.5:0.5b

curl http://127.0.0.1:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen2.5:0.5b",
    "stream": false,
    "messages": [
      {
        "role": "user",
        "content": "Classify: stop the robot"
      }
    ]
  }'
```

1. Check Docker

First check whether Docker is already installed and working:

```
docker --version
docker ps
```

If Docker works, skip to Step 3. If it is not installed, install Docker using the official apt repository method:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
```

```
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Enable and start Docker at boot:

```
sudo systemctl enable docker
sudo systemctl start docker
sudo docker run hello-world
```

Note for Ubuntu 18.04: if Docker is already installed and working, avoid reinstalling Docker unless necessary. Some current Docker documentation focuses on newer Ubuntu releases.

2. Optional: Configure NVIDIA GPU Support

Skip this step if `docker run --gpus all` already works on your machine. Otherwise, verify that the NVIDIA driver works on the host:

```
nvidia-smi
```

Install and configure NVIDIA Container Toolkit:

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -fsSL https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctl runtime configure --runtime=docker
sudo systemctl restart docker
```

Test GPU visibility from a container if needed:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

3. Handle an Existing Ollama Container

Check whether an Ollama container already exists:

```
docker ps -a | grep ollama
```

If it exists and you want to reuse it:

```
docker start ollama
docker update --restart unless-stopped ollama
docker logs -f ollama
```

If it exists but is broken, remove only the container. This does not delete the named model volume named ollama:

```
docker rm -f ollama
```

Do not run `docker volume rm ollama` unless you intentionally want to delete all downloaded models.

4. Create the Ollama Docker Container

GPU version - recommended for your desktop

```
docker run -d \
  --name ollama \
  --restart unless-stopped \
  --gpus all \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
  ollama/ollama
```

CPU-only fallback

```
docker run -d \
  --name ollama \
  --restart unless-stopped \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
  ollama/ollama
```

The important boot option is `--restart unless-stopped`. This makes Docker restart the container when Docker starts after a reboot. If you manually stop the container, Docker will respect that and keep it stopped until you start it again.

5. Confirm Ollama is Running

```
docker ps | grep ollama
curl http://127.0.0.1:11434/api/tags
```

If curl returns connection refused, inspect the logs:

```
docker logs ollama
```

6. Download the Models

Download the models you want available locally:

```
docker exec -it ollama ollama pull qwen2.5:0.5b
docker exec -it ollama ollama pull qwen2.5:1.5b
docker exec -it ollama ollama pull llama3.2:1b
```

Check installed models:

```
docker exec -it ollama ollama list
```

Model choice for your voice-command classifier:

- qwen2.5:0.5b - fastest and recommended starting point for STOP / SLOW_DOWN / PROCEED / UNKNOWN classification.
- qwen2.5:1.5b - better reasoning and classification accuracy, but slower.
- llama3.2:1b - another lightweight option; test latency against qwen2.5:0.5b.

7. Run a Model Interactively

```
docker exec -it ollama ollama run qwen2.5:0.5b
```

Other options:

```
docker exec -it ollama ollama run qwen2.5:1.5b
docker exec -it ollama ollama run llama3.2:1b
```

Exit the interactive session with:

```
/bye
```

8. Test the HTTP API

Use this API style from your ROS/Python classifier. For your existing ROS classifier, /api/chat is the safest endpoint to use with chat-style models:

```
curl http://127.0.0.1:11434/api/chat \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen2.5:0.5b",
  "stream": false,
  "messages": [
    {
      "role": "user",
      "content": "Classify this robot command as STOP, SLOW_DOWN, PROCEED, or UNKNOWN: stop
the robot right now"
    }
  ]
}'
```

Expected behavior: Ollama should return a JSON object containing the assistant response. Your ROS node can parse the response message content and normalize it to one of your labels.

9. Make Ollama Start After Reboot

Enable Docker itself at boot

```
sudo systemctl enable docker
sudo systemctl start docker
```

Enable the Ollama container restart policy

```
docker update --restart unless-stopped ollama
```

Verify the restart policy:

```
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' ollama
```

Expected output:

```
unless-stopped
```

Reboot test:

```
sudo reboot
```

After reboot, check:

```
docker ps | grep ollama  
curl http://127.0.0.1:11434/api/tags
```

10. Optional: Warm Up the Model Automatically After Boot

The container can be running after boot while the model is not loaded into memory yet. If you want the first response to be faster, create a warm-up service that sends an empty request after boot and keeps the model loaded.

Create the warm-up script:

```
sudo tee /usr/local/bin/ollama-warmup.sh > /dev/null <<'EOF'  
#!/usr/bin/env bash  
set -e  
  
MODEL="${1:-qwen2.5:0.5b}"  
  
for i in {1..60}; do  
    if curl -fsS http://127.0.0.1:11434/api/tags >/dev/null; then  
        break  
    fi  
    sleep 2  
done  
  
curl -fsS http://127.0.0.1:11434/api/generate \  
-H "Content-Type: application/json" \  
-d '{"model":"'${MODEL}',"prompt":"","stream":false,"keep_alive":"24h"}' \  
>/dev/null  
EOF  
  
sudo chmod +x /usr/local/bin/ollama-warmup.sh
```

Create a systemd service:

```
sudo tee /etc/systemd/system/ollama-warmup.service > /dev/null <<'EOF'
```

```
[Unit]
Description=Warm up Ollama model after boot
After=docker.service network-online.target
Requires=docker.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/ollama-warmup.sh qwen2.5:0.5b

[Install]
WantedBy=multi-user.target
EOF
```

Enable and start the warm-up service:

```
sudo systemctl daemon-reload
sudo systemctl enable ollama-warmup.service
sudo systemctl start ollama-warmup.service
```

Check status and loaded models:

```
systemctl status ollama-warmup.service
curl http://127.0.0.1:11434/api/ps
```

Use this warm-up only if you are okay with Ollama keeping memory occupied for the selected model.

11. Useful Maintenance Commands

```
# Start Ollama

docker start ollama

# Stop Ollama

docker stop ollama

# Restart Ollama

docker restart ollama

# View logs

docker logs -f ollama

# List models

docker exec -it ollama ollama list

# Remove a model

docker exec -it ollama ollama rm qwen2.5:1.5b
```

Update Ollama Docker image while keeping the model volume:

```
docker pull ollama/ollama
docker rm -f ollama
```

```
docker run -d \
  --name ollama \
  --restart unless-stopped \
  --gpus all \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
  ollama/ollama
```

12. Troubleshooting

Container name conflict

If you see a message saying the container name /ollama is already in use, you already have an Ollama container. Either start it or remove/recreate it:

```
# Reuse existing container

docker start ollama

# Or remove and recreate it

docker rm -f ollama
```

Connection refused on port 11434

This usually means the container is not running or Ollama failed to start:

```
docker ps -a | grep ollama
docker logs ollama
docker start ollama
curl http://127.0.0.1:11434/api/tags
```

GPU issue

If the GPU command fails, use the CPU-only Docker run command first to confirm Ollama works. Then fix NVIDIA Container Toolkit separately.

```
# CPU-only fallback

docker rm -f ollama

docker run -d \
  --name ollama \
  --restart unless-stopped \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
  ollama/ollama
```

Model not downloaded

```
docker exec -it ollama ollama list
docker exec -it ollama ollama pull qwen2.5:0.5b
```

13. Source References

Official Ollama Docker documentation: <https://docs.ollama.com/docker>

Ollama API documentation: <https://github.com/ollama/ollama/blob/main/docs/api.md>

Ollama qwen2.5 model library: <https://ollama.com/library/qwen2.5>

Ollama llama3.2 model library: <https://ollama.com/library/llama3.2>

Docker Ubuntu installation documentation: <https://docs.docker.com/engine/install/ubuntu/>

Docker restart policy documentation: <https://docs.docker.com/engine/containers/start-containers-automatically/>